Technical Interview Preparation Guide

Secure Note-Taking Application: Indexing, Aggregations & RBAC Architecture

Candidate: **Md Faisal Ahmed**
Role: **Full Stack / Backend Engineer**
Stack: **Node.js, Express, MongoDB, React, MUI**

1. MONGODB INDEXING STRATEGY & QUERY OPTIMIZATION

Q1 **Why did you define all indexes explicitly via `schema.index(...)` instead of inline in field definitions?**

Primary Technical Reasons:

- **Preventing Duplicate Index Warnings:** When declaring `unique: true` or `index: true` inline on schema paths in Mongoose while also declaring compound or custom indexes via `schema.index()`, Mongoose and MongoDB create redundant duplicate indexes or trigger duplicate schema index warnings.
- **Centralized Reviewability:** In enterprise code reviews, having all collection indexes declared at the bottom of the schema file via `schema.index(...)` gives developers and DBAs an instant snapshot of the collection's access patterns without scanning through nested property trees.
- **Strict Compliance:** The technical prompt explicitly mandated: *"You must use the schema.index method for defining indexes in your code so they are visible during review."*

> 💡 **Interview Pro-Tip:** Mention that Mongoose automatically ensures indexes on application startup when calling `Model.init()` or `mongoose.connect()` with `autoIndex: true`. In production high-traffic environments, you would disable `autoIndex` and manage indexes via DB migration scripts.

Q2 **Explain the compound index `{ userId: 1, createdAt: -1 }` on the `Note` collection. How does the Index Prefix Rule apply?**

When users fetch their notes, the REST API executes:

```
Note.find({ userId: req.user._id }).sort({ createdAt: -1 }).skip(skip).limit(limit)
```

How the Compound Index works:

- **Equality First, Sort Second (ESR Rule):** The index places `userId: 1` (Equality filter) first, immediately narrowing the search space to only notes owned by that specific user. The second key `createdAt: -1` provides pre-sorted order in reverse chronological order.
- **Eliminating In-Memory Sorting:** Without this compound index, MongoDB would have to load all user documents into RAM and perform an in-memory sort stage (`SORT` stage in `explain()`), which is capped at 32MB in MongoDB before throwing an error. With this index, the query execution plan achieves an optimal `IXSCAN` followed directly by a `FETCH`, with zero in-memory sorting.
- **Index Prefix Rule:** Because `userId` is the leading key (prefix) of `{ userId: 1, createdAt: -1 }`, MongoDB can use this compound index to satisfy any query that filters solely on `userId`. Therefore, creating a separate index on `{ userId: 1 }` is redundant and wasteful.

Q3 Why did you also create a standalone index `{ createdAt: -1 }` on the `Note` collection? Isn't it already in the compound index?

Answer: Yes, `createdAt` is in the compound index, but it is in the **second position**. Under MongoDB's Index Prefix Rule, an index can only be utilized if the query includes the index's prefix keys.

When an **Admin** accesses the system-wide notes monitor (`GET /api/admin/notes`), the query is:

```
Note.find().sort({ createdAt: -1 }).skip(skip).limit(limit)
```

Because there is **no `userId` filter**, MongoDB cannot utilize the `{ userId: 1, createdAt: -1 }` index. Without the standalone `{ createdAt: -1 }` index, MongoDB would perform a full collection scan (`COLLSCAN`) and an in-memory sort across all users' notes. Thus, `{ createdAt: -1 }` is strictly required and non-redundant.

Q4 What is a Multikey Index, and why was it necessary for `UserSchema.index({ interests: 1 })`?

In MongoDB, when an indexed field holds an array value (such as `interests: ['chess', 'reading', 'ai']`), MongoDB automatically creates a **Multikey Index**.

- For each document, MongoDB writes an index entry for *every individual element* in the array.
- In **Scenario 1 Aggregation**, the pipeline begins with `{ $unwind: "$interests" }` followed by `{ $group: { _id: "$interests", ... } }`. The multikey index allows the database to traverse distinct interest keys without scanning non-matching user profiles.

> ⚠️ **Constraint to Remember:** MongoDB does not allow a compound multikey index where more than one field is an array (e.g., cannot index two separate array fields together in one compound index).

Q5 Why did you index `{ userId: 1, createdAt: -1 }` on the `Post` collection for Scenario 2?

In **Scenario 2**, the aggregation pipeline joins the `User` collection with the `posts` collection using a `$lookup` stage:

```
{
  $lookup: {
    from: "posts",
    localField: "_id",
    foreignField: "userId",
    as: "posts"
  }
}
```

Under the hood, MongoDB executes an equality match on the foreign collection (`posts.userId == user._id`). Without an index on `userId` in the `posts` collection, MongoDB would perform a full collection scan of the entire `posts` collection for every user evaluated! By indexing `{ userId: 1, createdAt: -1 }` on `Post`, the `$lookup` stage becomes an indexed point lookup.

Q6 How do you verify index usage and query performance in MongoDB?

We use the `explain("executionStats")` method on any Mongoose or MongoDB query:

```
const stats = await Note.find({ userId }).sort({ createdAt: -1 }).explain("executionStats");
```

Key Metrics to Inspect:

Stage / Metric	Optimal Value	What It Signifies
`stage`	`IXSCAN` + `FETCH`	Uses index scan. If it shows `COLLSCAN`, an index is missing.
`totalKeysExamined`	Equal to returned docs	Number of index entries scanned.
`totalDocsExamined`	Equal to limit	Number of actual documents read from disk.
`executionTimeMillis`	< 5ms	Total execution latency.

2. MONGODB AGGREGATION PIPELINES DEEP-DIVE

Q7 Walk through Scenario 1 (Group by Interests). Why must exactly one `collection.aggregate()` call be used?

Code Implementation:

```
const result = await User.aggregate([
  // Stage 1: Unwind the array
  { $unwind: "$interests" },

  // Stage 2: Group by each individual interest and push matching users
  {
    $group: {
      _id: "$interests",
      totalUsers: { $sum: 1 },
      users: {
        $push: {
          _id: "$_id",
          username: "$username",
          email: "$email",
          role: "$role"
        }
      }
    }
  },

  // Stage 3: Alphabetical ordering by interest name
  { $sort: { _id: 1 } }
]);
```

Explanation:

- `$unwind`: Deconstructs the `interests` array in each user document, emitting one document per interest string. If Alice has `['chess', 'ai']`, two documents are produced for the pipeline.
- `$group`: Groups by `_id: "$interests"`, accumulating user sub-documents using the `$push` accumulator and counting matching users via `{ $sum: 1 }`.
- **Strict Constraint Met:** Accomplished completely in database engine memory within a single round-trip without application-level post-processing or multiple queries.

Q8 **Walk through Scenario 2 (User Posts with $lookup). Why did you aggregate from `User` instead of `Post`?**

Code Implementation:

```
const result = await User.aggregate([
  // Stage 1: Match the target user by primary key ObjectId
  { $match: { _id: new mongoose.Types.ObjectId(userId) } },

  // Stage 2: Join posts collection where foreignField 'userId' matches user '_id'
  {
    $lookup: {
      from: "posts",
      localField: "_id",
      foreignField: "userId",
      as: "posts"
    }
  },

  // Stage 3: Project public fields and suppress hashed password
  {
    $project: {
      _id: 1,
      username: 1,
      email: 1,
      role: 1,
      interests: 1,
      totalPosts: { $size: "$posts" },
      posts: 1
    }
  }
]);
```

Justification: The prompt specified: *"Retrieve all posts belonging to a particular user."* Initiating the pipeline from the `User` collection allows us to match the single user first via the primary index `_id` (fast single-doc point read), and then perform the `$lookup` against the indexed `Post` collection. This produces a cohesive entity representation with the user profile and their array of posts.

Q9 What is the 100MB RAM limit on MongoDB aggregations, and how do you handle it in production?

Answer: In MongoDB, aggregation pipeline stages (particularly `$group` and `$sort`) have a strict **100MB RAM limit** per stage by default. If exceeded, MongoDB aborts the query with an error.

Production Optimization Techniques:

1. **Early Filtering (`$match`):** Place `$match` stages as early as possible in the pipeline so subsequent stages process only a small subset of records.
2. **Early Projection (`$project`):** Discard unnecessary document fields before passing records to `$group` or `$sort`.
3. **Index-Backed Sorting:** Ensure `$sort` stages at the start of a pipeline are backed by an index so MongoDB doesn't have to buffer documents in memory.
4. `allowDiskUse: true`: When aggregating massive datasets (e.g. reporting or big data ETL), pass `{ allowDiskUse: true }` to permit temporary disk storage.

3. AUTHENTICATION, SECURITY & RBAC

Q10 Explain your password hashing approach. Why `bcryptjs` with 10 salt rounds?

Answer: Simple hash functions like MD5 or SHA-256 are fast, which makes them vulnerable to brute-force attacks and pre-computed rainbow table lookups on GPUs.

- **Bcrypt is Adaptive:** It uses an adaptive key derivation algorithm based on the Blowfish cipher. The work factor (salt rounds) determines how many hashing rounds ($2^{10} = 1024$ rounds for salt 10) are executed.
- **Salt Generation:** A unique cryptographic salt is generated for each password, ensuring that identical passwords produce completely different hash strings.
- **Mongoose Pre-Save Hook:**

```
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});
```

The `isModified('password')` guard ensures the password is only re-hashed when the user actually changes their password, preventing accidental double-hashing on profile updates.

Q11 Why is `password` marked with `select: false` in the User schema?

Answer: In Mongoose, `select: false` ensures that any standard query (e.g. `User.find()`, `User.findById()`, or queries in aggregations) automatically excludes the password field from the query projection.

This provides defense-in-depth against accidental data leakage in API responses, logs, or serialization. When authentication needs to verify a password during login, we explicitly opt-in using `User.findOne({ email }).select('+password')`.

Q12 How does your Role-Based Access Control (RBAC) middleware work?

We implemented a two-tier middleware architecture:

1. `protect` **middleware ([`auth.js`] (file:///c:/Users/User/Documents/GitHub/assignment/src/middleware/auth.js)):** Extracts and verifies the JWT token from the `Authorization: Bearer <token>` header. Fetches the active user from MongoDB (verifying the account still exists) and attaches it to `req.user`.
2. `authorize(...roles)` **middleware ([`rbac.js`] (file:///c:/Users/User/Documents/GitHub/assignment/src/middleware/rbac.js)):** Higher-order middleware that checks whether `req.user.role` is in the allowed roles array. If not, it rejects the request with HTTP `403 Forbidden`.

In controller methods (e.g. `getNoteById`, `updateNote`, `deleteNote`), role inheritance is enforced:

```
const isOwner = note.userId.toString() === req.user._id.toString();
const isAdmin = req.user.role === 'admin';
if (!isOwner && !isAdmin) {
  return res.status(403).json({ success: false, message: 'Forbidden' });
}
```

4. LIVE CODING & REAL-WORLD INTERVIEW SCENARIOS

Q13 **Live Coding Challenge: Write an aggregation pipeline to find the Top 3 users with the highest count of posts.**

Solution:

```
const topPosters = await Post.aggregate([
  // Stage 1: Group by userId and count total posts
  {
    $group: {
      _id: "$userId",
      postCount: { $sum: 1 }
    }
  },
  // Stage 2: Sort by postCount descending
  { $sort: { postCount: -1 } },
  // Stage 3: Limit to top 3
  { $limit: 3 },
  // Stage 4: Lookup user profile details
  {
    $lookup: {
      from: "users",
      localField: "_id",
      foreignField: "_id",
      as: "user"
    }
  },
  // Stage 5: Unwind user array
  { $unwind: "$user" },
  // Stage 6: Project clean output
  {
    $project: {
      _id: 1,
      username: "$user.username",
      email: "$user.email",
      postCount: 1
    }
  }
]);
```

Index Backing: Utilizes `PostSchema.index({ userId: 1, createdAt: -1 })` for fast grouping.

Q14 Live Coding Challenge: If a user searches for notes containing the word "architecture" in the title or content, what index would you propose?

Answer: Regular regex searches like `Note.find({ title: /architecture/i })` cannot utilize standard B-tree indexes efficiently because of the leading wildcard, leading to expensive `COLLSCAN` operations.

Optimal Solution: A Text Index

```
noteSchema.index({ title: 'text', content: 'text' }, { weights: { title: 3, content: 1 } });
```

Query Execution:

```
Note.find(
  { $text: { $search: "architecture" }, userId: req.user._id },
  { score: { $meta: "textScore" } }
).sort({ score: { $meta: "textScore" } });
```

This leverages MongoDB's inverted index, providing tokenization, stemming, stop-word elimination, and relevance scoring.

Q15 How does your system handle pagination at scale? When would you use Keyset (Cursor) over Skip/Limit?

In this assignment, we implemented standardized offset pagination (`skip` and `limit`):

```
const skip = (page - 1) * limit;
Note.find({ userId }).sort({ createdAt: -1 }).skip(skip).limit(limit);
```

Limitation of Offset Pagination: As the offset increases (e.g. `skip(50000)`), MongoDB must scan 50,010 index entries and discard the first 50,000 before returning 10. This creates $O(N)$ scanning latency.

Keyset / Cursor-Based Pagination Alternative:

```
Note.find({ userId, createdAt: { $lt: lastSeenCreatedAt } })
  .sort({ createdAt: -1 })
  .limit(limit);
```

Because the query uses an inequality filter on the indexed `createdAt` field, MongoDB jumps directly to the target leaf node in the B-tree in $O(\log N)$ time, achieving constant latency regardless of whether you are viewing page 1 or page 10,000.